

Alcance de la tecnología: la infraestructura como código

Funcionamiento de un sistema completo, capas y módulos principales, y modelo de objetos y procesos

Carlos Luis Rojas Aragonés
Gestión del Desarrollo Tecnológico

27 de agosto de 2026

Resumen

La infraestructura como código (*Infrastructure as Code*, IaC) sustituye la configuración manual de servidores, redes y servicios por una especificación versionada que un motor automático traduce en operaciones sobre la interfaz de programación de un proveedor. Este trabajo delimita el alcance de la tecnología, describe su principio de funcionamiento (la conciliación continua entre un estado deseado y un estado real), desglosa las siete capas y los seis módulos transversales de un sistema completo, y formaliza el conjunto mediante un modelo de objetos y procesos según la metodología OPM, con su diagrama de sistema, sus enlaces estructurales, su ampliación en detalle y el lenguaje OPL correspondiente. Todas las figuras son de elaboración propia.

1. Objeto y delimitación del alcance

01. El encargo

Esta primera parte de la actividad pide tres cosas: un resumen del funcionamiento básico de un sistema completo de infraestructura como código, un conjunto de imágenes y diagramas que expliquen sus capas o módulos principales, y un diagrama de proceso-objeto elaborado en el entorno OPM Sandbox (Enterprise Systems Modeling Laboratory, Technion, s.f.). El documento atiende las dos primeras en las secciones 2 a 8, con el resumen funcional y los diagramas de capas y módulos, y la tercera en la sección 9, que contiene el modelo de objetos y procesos con su diagrama de sistema, sus enlaces estructurales, su ampliación en detalle y el lenguaje OPL correspondiente. El diagrama construido en el propio entorno de la herramienta, que no guarda ni exporta el trabajo, se entrega aparte.

02. Qué se entiende aquí por sistema completo

Conviene fijar el término antes de dibujar nada, porque «infraestructura como código» se usa con dos alcances muy distintos. En el sentido estrecho designa una herramienta: un fichero declarativo y un binario que lo aplica. En el sentido amplio, que es el que aquí interesa, designa un sistema sociotécnico completo, con su cadena de herramientas, su modelo de estado, sus controles de gobierno y el equipo que lo opera.

Un sistema completo de IaC es aquel que satisface cuatro propiedades simultáneas (Morris, 2021):

1. **Especificación externalizada.** La configuración deseada de la infraestructura vive fuera de la infraestructura, en artefactos de texto legibles y versionados, no en la memoria de un operador ni en el estado acumulado de las máquinas.
2. **Reproducibilidad.** Aplicar la misma especificación sobre un sustrato vacío produce un resultado equivalente, con independencia de quién la aplique y de cuándo lo haga.
3. **Idempotencia.** Aplicar la especificación sobre un sistema que ya la cumple no produce ningún cambio, y aplicarla n veces tiene el mismo efecto que aplicarla una vez.
4. **Conciliación.** El sistema es capaz de detectar la divergencia entre lo especificado y lo realmente desplegado, y de corregirla.

La literatura académica sobre la materia, todavía escasa en comparación con su difusión industrial, se ha concentrado sobre todo en la calidad y los defectos del propio código de infraestructura (Rahman, Mahdavi-Hezaveh & Williams, 2019).

Un montaje que cumpla sólo la primera propiedad es un repositorio de guiones, no un sistema de IaC. La cuarta es la que convierte la automatización en control, y es también la que con más frecuencia falta en las implantaciones reales.

03. La frontera del sistema

Delimitar el alcance exige decir también qué queda fuera. El Cuadro 1 fija la frontera adoptada en este documento.

Cuadro 1: Frontera del sistema de infraestructura como código.

Dentro del alcance	Fuera del alcance
Cómputo, red, almacenamiento, identidad y servicios gestionados declarados como código.	El sustrato físico del proveedor: centros de datos, cableado, refrigeración.
Configuración del sistema operativo y de los agentes instalados en él.	El código de la aplicación de negocio y sus pruebas funcionales.
Construcción de imágenes y artefactos inmutables.	Los datos de producción y su ciclo de vida, que exigen migraciones y copias de seguridad con reglas propias.
Orquestación de cargas y despliegue continuo de manifiestos.	La gestión de incidencias, la guardia y los procedimientos de crisis.
Estado, secretos, políticas, pruebas y observabilidad de la propia infraestructura.	La observabilidad funcional de la aplicación (trazas de negocio, experiencia de usuario).

La distinción entre infraestructura y datos merece un comentario, porque es la frontera que más problemas causa en la práctica. Un servidor de base de datos está dentro del alcance: su tamaño, su versión, su red y sus copias programadas se declaran como código. El contenido de esa base de datos no lo está: no es idempotente, no se puede destruir y recrear, y su presencia obliga a que el sistema de IaC distinga recursos desechables de recursos con estado persistente, y proteja a los segundos de las operaciones de reemplazo.

2. El principio de funcionamiento

01. Tres estados y una función

Todo el funcionamiento de un sistema de IaC se sostiene sobre tres representaciones de la misma infraestructura y sobre la función que las compara.

Estado deseado.

Lo que la especificación declara. Es un artefacto de texto, versionado, que describe recursos y sus propiedades. No describe los pasos para llegar a ellos.

Estado registrado.

Lo que el motor cree que existe, resultado de la última operación que él mismo ejecutó. Se materializa en un fichero o servicio de estado que asocia cada recurso declarado con el identificador real que el proveedor le asignó (HashiCorp, s.f.).

Estado real.

Lo que efectivamente existe en el proveedor, que sólo se conoce consultando su interfaz de programación.

La función central del sistema es la **conciliación**: calcular la diferencia entre el estado deseado y el estado real, y emitir el conjunto mínimo de operaciones que anula esa diferencia. La Figura 1 representa el bucle.

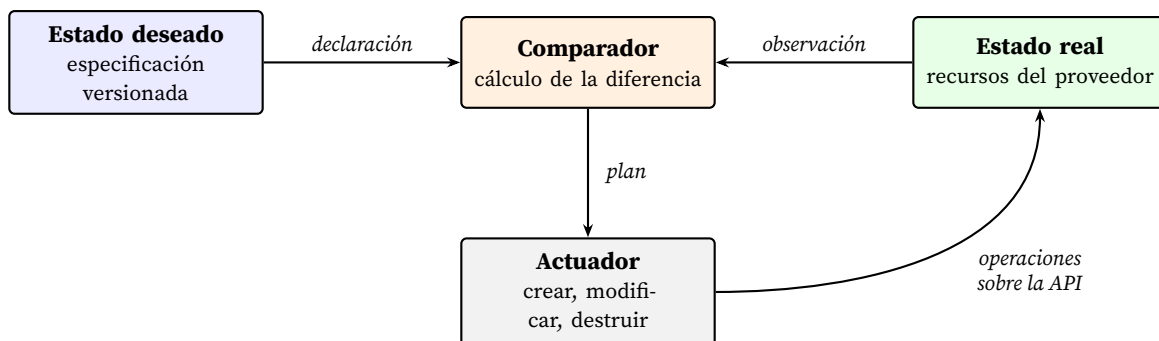


Figura 1: Bucle de conciliación. El sistema no ejecuta una secuencia de pasos, sino que cierra repetidamente la distancia entre dos representaciones. Si la diferencia es vacía, el actuador no ejecuta nada, y ésta es exactamente la propiedad de idempotencia. Elaboración propia.

02. Declarativo frente a imperativo

La diferencia entre los dos paradigmas no es de sintaxis sino de quién calcula el camino. En el enfoque imperativo, el autor escribe la secuencia de operaciones y asume la responsabilidad de que sea correcta desde cualquier estado de partida. En el enfoque declarativo, el autor escribe el destino y el motor deriva la secuencia.

La consecuencia práctica es que un guion imperativo sólo es correcto para el estado de partida que su autor imaginó, mientras que una especificación declarativa es correcta para cualquiera, a condición de que el motor sepa observar el estado real. Por eso el paradigma declarativo domina la capa de aprovisionamiento, donde el espacio de estados de partida es enorme, y sobrevive el imperativo en tareas de configuración muy locales, donde ese espacio es pequeño y conocido.

Casi ningún sistema es puro. Las herramientas de configuración expresan los pasos en orden, pero cada paso es un módulo idempotente que sólo actúa si hace falta (Red Hat, s.f.). Es un híbrido: secuencia imperativa de operaciones declarativas.

03. Convergencia y deriva

Un sistema convergente es el que, aplicado repetidamente, se aproxima al estado deseado y luego se queda quieto. La **deriva** (*drift*) es la aparición de diferencias entre el estado real y el deseado por causas ajenas al sistema: un cambio manual de urgencia, una acción automática del proveedor, la caducidad de un certificado, el borrado accidental de un recurso.

La deriva es inevitable, y la madurez de una implantación se mide por lo que hace con ella. Hay tres posturas, en orden creciente de rigor:

1. **Ignorarla.** El motor sólo se ejecuta cuando alguien lo lanza. Entre ejecuciones, el estado real puede alejarse arbitrariamente.
2. **Detectarla.** Una ejecución periódica en modo de sólo lectura compara y notifica, sin corregir. Convierte la deriva en una métrica observable.
3. **Corregirla.** Un agente concilia de forma continua y revierte todo cambio no declarado. Es el principio de conciliación continua de GitOps (OpenGitOps, s.f.) y el modo nativo de los controladores de Kubernetes (Burns et al., 2016; The Kubernetes Authors, s.f.).

3. El ciclo básico de operación

Descendiendo un nivel desde el bucle abstracto, la operación cotidiana de un sistema completo recorre siete pasos. La Figura 2 los ordena.

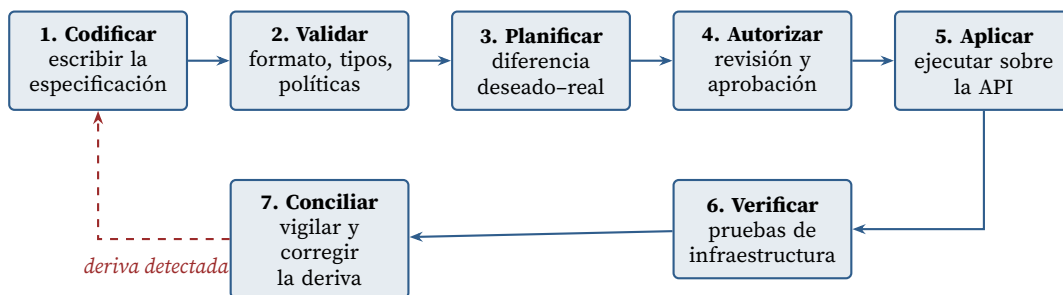


Figura 2: Los siete pasos del ciclo de operación. El retorno discontinuo es el que distingue un sistema de IaC de una simple automatización de despliegue. Elaboración propia.

Los pasos 2, 4 y 6 son los que suelen faltar en implantaciones incompletas, y son precisamente los que aportan garantía. Sin validación de políticas, el sistema automatiza también los errores de configuración; sin autorización, elimina el punto de control humano sobre cambios irreversibles; sin verificación, no distingue entre «el motor terminó sin error» y «la infraestructura funciona».

01. El paso crítico: la comparación a tres bandas

El paso 3 merece detalle porque en él se concentra el mecanismo que hace útil al sistema. El motor no compara dos cosas sino tres, y lo hace en dos fases (Figura 3).

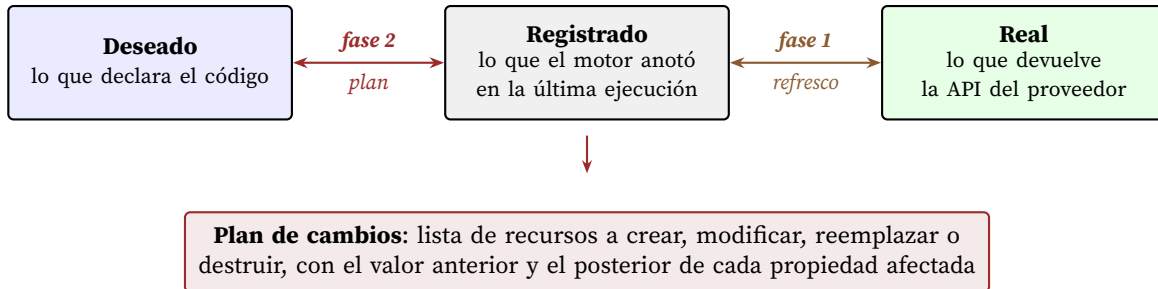


Figura 3: Comparación a tres bandas. La fase de refresco actualiza el registro contra el mundo real y es la que descubre la deriva; la fase de plan compara el código contra ese registro ya actualizado y es la que produce las operaciones. Elaboración propia.

El estado registrado no es redundante. Cumple tres funciones que ninguna de las otras dos representaciones puede cumplir. Primera, traduce el nombre lógico que el código da a un recurso al identificador opaco que el proveedor le asignó, sin lo cual el motor no podría reconocer que un recurso existente le pertenece. Segunda, permite detectar destrucciones: un recurso presente en el registro y ausente del proveedor sólo puede detectarse comparando contra el registro, nunca comparando el código con la realidad. Tercera, guarda propiedades calculadas por el proveedor que el código no declara y que otros recursos necesitan.

Ese fichero es, en consecuencia, un activo crítico: contiene identificadores y con frecuencia valores sensibles, y su corrupción o pérdida deja al sistema sin capacidad de reconocer lo que él mismo creó. De ahí que todo montaje serio lo almacene en un servicio remoto con cifrado, versionado y bloqueo de concurrencia.

4. Las capas de un sistema completo

Un sistema completo se organiza en siete capas apiladas, cada una de las cuales consume el resultado de la anterior y expone una abstracción más alta. La Figura 4 las presenta con sus herramientas representativas.

01. Qué hace cada capa

L0, sustrato. No es parte del sistema de IaC sino su objeto. Aporta la interfaz de programación sobre la que todo lo demás actúa. Su característica determinante es que ofrece recursos con identidad propia, creables y destruibles por llamada, lo que es la condición de posibilidad de todo el resto.

L1, aprovisionamiento. Traduce la especificación en llamadas de creación, modificación y destrucción. Es la capa que posee el registro de estado y la que ejecuta la comparación a tres bandas. Trabaja con recursos que todavía no tienen sistema operativo arrancado, o cuyo interior no le concierne. Conviene señalar que en 2023 el cambio de licencia de Terraform dio origen a OpenTofu,

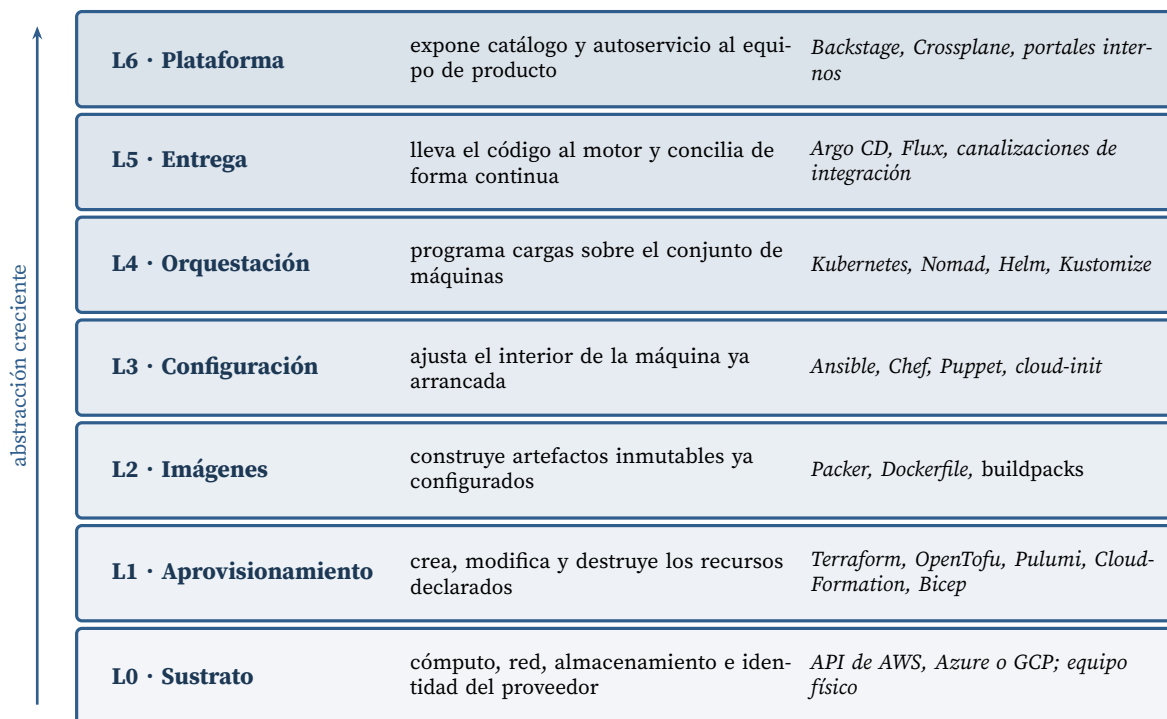


Figura 4: Las siete capas de un sistema completo de infraestructura como código. Cada capa consume la salida de la inferior. Las herramientas citadas son representativas, no exhaustivas. Elaboración propia.

una bifurcación acogida por la Linux Foundation (The Linux Foundation, 2023), de modo que hoy la capa cuenta con dos implementaciones compatibles del mismo lenguaje: una decisión de dependencia que una hoja de ruta tecnológica debería registrar de forma explícita.

L2, imágenes. Desplaza el trabajo de configuración desde el momento de ejecución al momento de construcción. En lugar de arrancar una máquina genérica y modificarla, construye una imagen ya completa y arranca instancias idénticas de ella. Es la base de la infraestructura inmutable: en lugar de modificar un servidor, se reemplaza. Reduce drásticamente la deriva, porque elimina la ventana temporal en la que un servidor puede divergir.

L3, configuración. Actúa sobre el interior de máquinas ya arrancadas: paquetes, ficheros, servicios, usuarios. Es la capa históricamente más antigua y la que más terreno ha cedido a L2 y L4. Sigue siendo necesaria donde hay máquinas de larga vida, sistemas heredados o equipos físicos.

L4, orquestación. Introduce una indirección importante: deja de declararse «esta carga en esta máquina» y pasa a declararse «esta carga, tantas réplicas» sobre un conjunto de máquinas intercambiables. El planificador decide la colocación. Sus controladores son bucles de conciliación como el de la Figura 1, pero embebidos y permanentes (The Kubernetes Authors, s.f.).

L5, entrega. Conecta el repositorio con el motor. Ejecuta las canalizaciones que validan, planifican y aplican, y, en el modelo de conciliación continua, aloja el agente que vigila la deriva sin intervención humana (Humble & Farley, 2010; OpenGitOps, s.f.).

L6, plataforma. Envuelve todo lo anterior en una interfaz de autoservicio, para que un equipo de producto pueda solicitar una base de datos sin conocer ni la capa L1 ni las políticas de la

organización, que quedan codificadas en el catálogo.

02. La tensión entre capas

Las capas L2, L3 y L4 compiten parcialmente por el mismo territorio. Un mismo requisito, por ejemplo «que este servidor tenga la versión 1.24 de un tiempo de ejecución», puede resolverse construyendo una imagen que ya la incluya (L2), instalándola con un módulo de configuración (L3) o empaquetando la aplicación en un contenedor que la lleve dentro (L4). Las tres soluciones funcionan y no deben coexistir sobre el mismo recurso: si dos capas gobiernan la misma propiedad, cada una revierte el trabajo de la otra en cada ciclo de conciliación, y el sistema oscila.

Decidir qué capa posee cada propiedad es, por tanto, la primera decisión de arquitectura de una implantación, y conviene documentarla de forma explícita.

5. Los módulos transversales

Las siete capas describen el flujo principal, pero un sistema completo necesita además seis módulos que atraviesan todas las capas (Figura 5). Ninguno produce infraestructura; todos condicionan si la que se produce es correcta, segura y auditable.

El Cuadro 2 resume la función de cada uno y el fallo característico que aparece cuando falta.

Cuadro 2: Módulos transversales: función, instrumentos y fallo característico por ausencia.

Módulo	Función	Fallo por ausencia
Control de versiones	Registrar quién cambió qué, cuándo y por qué; permitir revertir a un estado anterior conocido.	Cambios sin trazabilidad y reversión imposible.
Gestión de estado	Custodiar el registro: almacenamiento remoto, cifrado, versionado y bloqueo para impedir ejecuciones concurrentes.	Corrupción del registro y recursos huérfanos.
Gestión de secretos	Mantener credenciales fuera del código e inyectarlas en el momento de la ejecución, con rotación y ámbito mínimo. Es uno de los defectos más frecuentes en código de infraestructura (Rahman, Parnin & Williams, 2019).	Credenciales escritas en el repositorio.
Política como código	Expresar las reglas de la organización como predicados evaluables sobre el plan, antes de aplicarlo (Open Policy Agent, s.f.).	El sistema automatiza también los incumplimientos, y a mayor velocidad.
Pruebas	Verificar la especificación en varios niveles: análisis estático, validación del plan, despliegue efímero y comprobación funcional.	«Terminó sin error» se confunde con «funciona».
Observabilidad	Medir deriva, coste, tiempo de ciclo y cumplimiento, y exponerlos como series temporales.	No hay figuras de mérito y la mejora no es demostrable.

6. Modos de operación

Sobre la misma arquitectura caben decisiones de operación que cambian de forma apreciable el comportamiento del sistema. Las tres principales son independientes entre sí.

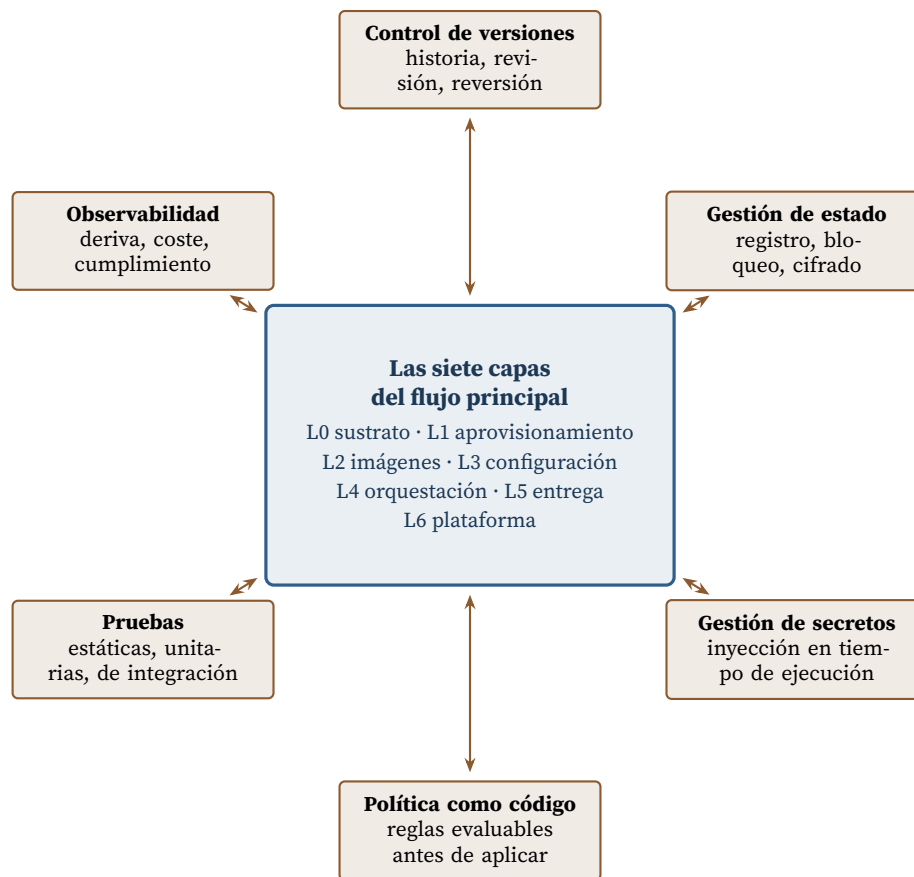


Figura 5: Los seis módulos transversales. Cada uno interactúa con todas las capas del flujo principal, no con una en particular. Elaboración propia.

01. Empuje frente a arrastre

En el modo de **empuje** (*push*), un actor externo, típicamente una canalización de integración, posee las credenciales del entorno y ejecuta el motor contra él. En el modo de **arrastre** (*pull*), un agente residente dentro del entorno consulta el repositorio y se aplica a sí mismo los cambios. La Figura 6 contrasta ambos.

El arrastre tiene dos ventajas estructurales. La primera es de seguridad: las credenciales de producción no viajan a un sistema de integración compartido. La segunda es que el agente, por estar siempre presente, puede conciliar de forma continua y no sólo cuando alguien confirma un cambio, que es la cuarta propiedad exigida en la sección 1. Su coste es que exige un agente por entorno y complica los flujos que necesitan orden entre entornos.

02. Mutable frente a inmutable

En el modelo mutable, un recurso desplegado se modifica en su sitio. En el inmutable, no se modifica nunca: se construye uno nuevo con la configuración deseada y se sustituye. El modelo inmutable elimina por construcción la deriva de configuración y hace que el recurso desplegado sea siempre trazable a un artefacto concreto. A cambio, obliga a que todo estado persistente viva fuera del

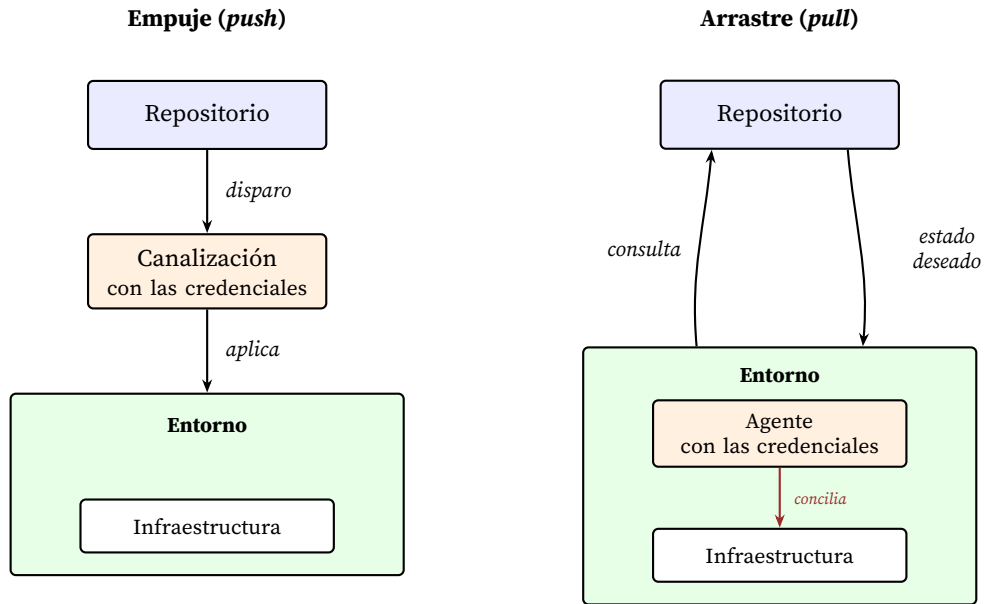


Figura 6: Empuje frente a arrastre. En el arrastre, las credenciales del entorno nunca salen de él, lo que reduce la superficie de ataque y hace del agente el punto natural de conciliación continua. Elaboración propia a partir de los principios de OpenGitOps (s.f.).

recurso reemplazable, lo que es una restricción de diseño exigente.

03. El cuadro de decisión

Las tres decisiones son independientes entre sí, de modo que caben ocho combinaciones. La más frecuente en implantaciones recientes es la de arrastre, inmutable y declarativa, pero ninguna de las tres opciones es obligada. El Cuadro 3 resume las consecuencias de cada una.

Cuadro 3: Tres decisiones de operación y sus consecuencias.		
Decisión	Opción A	Opción B
Iniciativa	Empuje: control de orden entre entornos; credenciales fuera del entorno; conciliación sólo bajo disparo.	Arrastre: credenciales confinadas; conciliación continua; un agente por entorno.
Mutabilidad	Mutable: cambios rápidos y baratos; deriva acumulativa; difícil trazar qué versión corre.	Inmutable: sin deriva; reemplazo trazable; obliga a externalizar todo estado persistente.
Expresión	Declarativa: el motor calcula el camino; correcta desde cualquier estado de partida.	Imperativa: control fino del orden; sólo correcta desde el estado de partida previsto.

7. El módulo como unidad de composición

El término «módulo» tiene en IaC un sentido preciso: es la unidad reutilizable que encapsula un conjunto de recursos tras una interfaz de entradas y salidas. Es el mecanismo que permite que una

organización no repita la misma configuración en cada equipo y, sobre todo, que las decisiones correctas queden incorporadas por omisión.

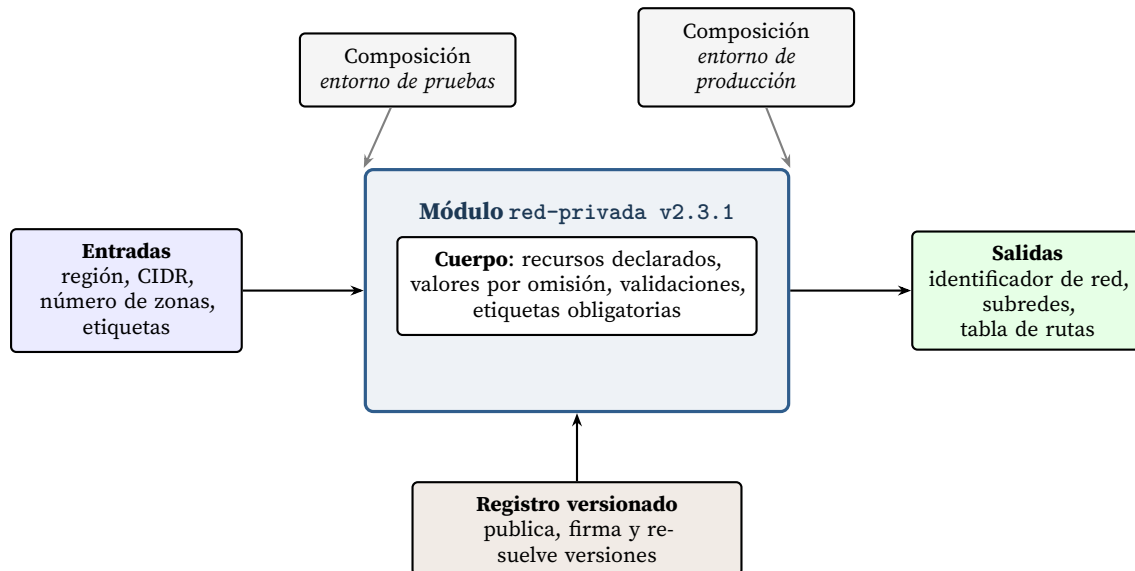


Figura 7: Anatomía de un módulo. Las mismas entradas con valores distintos producen entornos comparables; el registro versionado permite que una corrección se propague de forma controlada. Elaboración propia.

Tres propiedades hacen que un módulo cumpla su función. Debe tener una **interfaz estrecha**: cuantas menos entradas expone, más decisiones fija y más valor aporta. Debe estar **versionado** con semántica explícita, para que quien lo consume elija cuándo adoptar un cambio. Y debe delimitar un **radio de explosión** razonable: un módulo que agrupa demasiados recursos obliga a que cada cambio pequeño planifique sobre un conjunto grande, lo que alarga el ciclo y aumenta el riesgo de cada aplicación.

8. Figuras de mérito del sistema

Un sistema de IaC es en sí mismo una tecnología, y como tal admite figuras de mérito que permiten situarlo y seguir su evolución. El Cuadro 4 propone ocho, con su unidad y su procedimiento de medida. Las cuatro primeras coinciden con las métricas de rendimiento de entrega ampliamente utilizadas (Forsgren et al., 2018); las cuatro últimas son específicas de la infraestructura.

El tiempo de reconstrucción es la figura más reveladora de todas, porque verifica de un solo golpe las cuatro propiedades exigidas en la sección 1. Una organización que no puede reconstruir un entorno completo desde el repositorio no tiene un sistema de IaC completo, con independencia de cuántos ficheros declarativos posea.

Cuadro 4: Figuras de mérito propuestas para un sistema de IaC.

Figura de mérito	Unidad	Cómo se mide
Tiempo de ciclo del cambio	horas	Del registro del cambio en el repositorio a su aplicación efectiva en producción.
Frecuencia de aplicación	cambios/semana	Recuento de aplicaciones correctas por entorno.
Tasa de fallo del cambio	%	Aplicaciones que exigen reversión o corrección inmediata sobre el total.
Tiempo de restablecimiento	minutos	De la detección del fallo a la recuperación del servicio.
Cobertura de codificación	%	Recursos gestionados por el motor sobre el total de recursos existentes en el proveedor.
Tasa de deriva	recursos desviados / 1000 · semana	Diferencias detectadas en las ejecuciones de sólo lectura.
Tiempo de reconstrucción	horas	De un entorno vacío a un entorno equivalente al de producción, partiendo sólo del repositorio.
Cumplimiento de política	%	Planes que superan la evaluación de políticas sin excepción manual.

9. Modelo de objetos y procesos (OPM)

01. Por qué OPM para este sistema

La metodología de objetos y procesos, normalizada como ISO/PAS 19450:2015 y posteriormente como ISO 19450:2024 (Dori, 2002, 2016; International Organization for Standardization, 2015, 2024), presenta dos rasgos que la hacen adecuada aquí. El primero es que usa un único tipo de diagrama para representar a la vez estructura y comportamiento, lo que evita el problema habitual de mantener sincronizados un diagrama de bloques y un diagrama de actividad. El segundo es su carácter bimodal: cada diagrama tiene una traducción automática a lenguaje natural controlado (OPL), lo que permite verificar el modelo leyéndolo.

Para un sistema de IaC, cuya esencia es un proceso que transforma el estado de un objeto (la infraestructura desplegada) usando otros objetos como instrumentos, la correspondencia con el vocabulario de OPM es casi directa.

02. Entidades del diagrama de sistema

El Cuadro 5 enumera las entidades del nivel superior.

La notación se reparte en dos figuras para que ninguna quede saturada. La Figura 8 recoge los enlaces procedimentales, es decir, los que unen el proceso con los objetos que lo habilitan y con los que transforma: círculo hueco para instrumento, círculo relleno para agente, flecha doble para efecto y flecha simple para resultado. La Figura 9 recoge los enlaces estructurales del objeto *Especificación de Infraestructura*: triángulo relleno para agregación, triángulo hueco para generalización y triángulo doble para exhibición de un atributo. En ambas, el rectángulo denota objeto, la elipse denota proceso y el rectángulo de esquinas redondeadas denota estado.

Cuadro 5: Entidades del diagrama de sistema (SD) del modelo OPM.

Entidad	Tipo	Papel en el modelo
Aprovisionamiento de Infraestructura	Proceso	Proceso principal del sistema; se amplía en detalle en el diagrama SD1.
Equipo de Plataforma	Objeto (físico)	Agente humano que maneja el proceso.
Especificación de Infraestructura	Objeto	El código; instrumento del proceso. Se especializa en módulo reutilizable y configuración de entorno, y exhibe el atributo Versión.
Repositorio Versionado	Objeto	Todo del que la especificación es parte.
Motor de IaC	Objeto	Instrumento que ejecuta la conciliación.
Registro de Estado	Objeto	Objeto afectado: el proceso lo lee y lo reescribe en cada aplicación.
Proveedor de Nube	Objeto (físico)	Instrumento: expone la interfaz de programación.
Credenciales	Objeto	Instrumento con el que el motor se autentica.
Conjunto de Políticas	Objeto	Instrumento de la validación.
Infraestructura Desplegada	Objeto (físico)	Operando del sistema. Estados: inexistente, conforme, desviada, retirada.
Plan de Cambios	Objeto	Resultado del proceso, consumido por la aplicación.
Registro de Auditoría	Objeto	Resultado del proceso.

03. El lenguaje OPL del diagrama de sistema

La traducción del diagrama al lenguaje de objetos y procesos permite verificarlo por lectura. Las palabras reservadas van en versalitas.

EQUIPO DE PLATAFORMA es físico.
 INFRAESTRUCTURA DESPLEGADA es física.
 PROVEEDOR DE NUBE es físico y externo.
 REPOSITORIO VERSIONADO consta de ESPECIFICACIÓN DE INFRAESTRUCTURA.
 ESPECIFICACIÓN DE INFRAESTRUCTURA puede ser MÓDULO REUTILIZABLE o CONFIGURACIÓN DE ENTORNO.
 ESPECIFICACIÓN DE INFRAESTRUCTURA exhibe VERSIÓN.
 INFRAESTRUCTURA DESPLEGADA puede ser inexistente, conforme, desviada o retirada.
 APROVISIONAMIENTO DE INFRAESTRUCTURA requiere ESPECIFICACIÓN DE INFRAESTRUCTURA, MOTOR DE IAC, CONJUNTO DE POLÍTICAS, PROVEEDOR DE NUBE y CREDENCIALES.
 EQUIPO DE PLATAFORMA maneja APROVISIONAMIENTO DE INFRAESTRUCTURA.
 APROVISIONAMIENTO DE INFRAESTRUCTURA afecta a REGISTRO DE ESTADO.
 APROVISIONAMIENTO DE INFRAESTRUCTURA cambia INFRAESTRUCTURA DESPLEGADA de inexistente a conforme.
 APROVISIONAMIENTO DE INFRAESTRUCTURA produce PLAN DE CAMBIOS y REGISTRO DE AUDITORÍA.

Leído en voz alta, el conjunto de frases es una descripción funcional completa del sistema, y cualquier error de modelado se manifiesta como una frase falsa.

04. Ampliación en detalle del proceso principal

El diagrama SD1 amplía el proceso principal en siete subprocesos, ordenados verticalmente según la convención de OPM, en la que la posición vertical denota precedencia temporal. Los objetos que

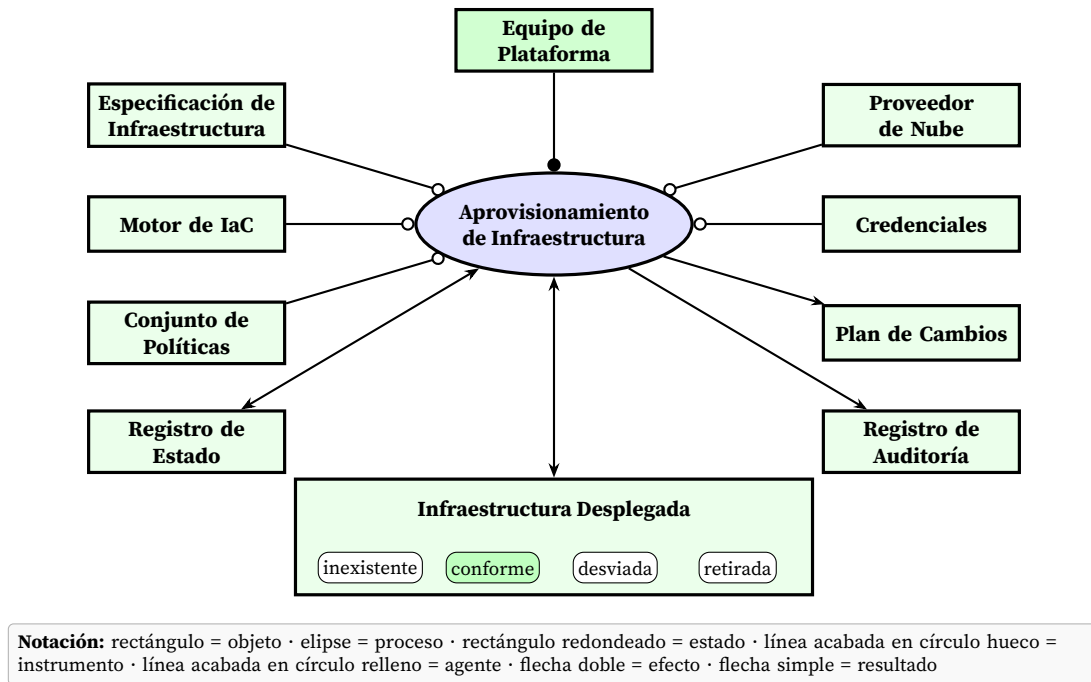


Figura 8: Diagrama de sistema (SD) del sistema de infraestructura como código, en notación OPM. Elaboración propia siguiendo Dori (2016) y International Organization for Standardization (2024).

sólo intervienen en el interior del proceso se dibujan dentro de la elipse; el *Equipo de Plataforma*, que pertenece al entorno, queda fuera. La Figura 10 lo recoge.

05. El lenguaje OPL de la ampliación

APROVISIONAMIENTO DE INFRAESTRUCTURA consta de CODIFICACIÓN, VALIDACIÓN, PLANIFICACIÓN, AUTORIZACIÓN, APLICACIÓN, CONCILIACIÓN y RETIRADA.
EQUIPO DE PLATAFORMA maneja CODIFICACIÓN y AUTORIZACIÓN.
CODIFICACIÓN produce ESPECIFICACIÓN DE INFRAESTRUCTURA.
VALIDACIÓN requiere CONJUNTO DE POLÍTICAS.
VALIDACIÓN cambia ESPECIFICACIÓN DE INFRAESTRUCTURA de borrador a validada.
PLANIFICACIÓN requiere REGISTRO DE ESTADO y ESPECIFICACIÓN DE INFRAESTRUCTURA.
PLANIFICACIÓN produce PLAN DE CAMBIOS.
AUTORIZACIÓN cambia PLAN DE CAMBIOS de pendiente a aprobado.
APLICACIÓN requiere PLAN DE CAMBIOS, CREDENCIALES y PROVEEDOR DE NUBE.
APLICACIÓN cambia INFRAESTRUCTURA DESPLEGADA de inexistente a conforme.
APLICACIÓN afecta a REGISTRO DE ESTADO.
CONCILIACIÓN cambia INFRAESTRUCTURA DESPLEGADA de desviada a conforme.
CONCILIACIÓN produce REGISTRO DE AUDITORÍA.
RETIRADA cambia INFRAESTRUCTURA DESPLEGADA de conforme a retirada.

Merece la pena señalar lo que el modelo hace explícito y que las descripciones en prosa suelen dejar implícito. Primero, que REGISTRO DE ESTADO es el único objeto que aparece a la vez como instrumento de la planificación y como objeto afectado por la aplicación, lo que identifica exactamente por qué necesita bloqueo de concurrencia. Segundo, que CONCILIACIÓN y APLICACIÓN cambian el mismo objeto desde estados de partida distintos, que es la formulación precisa de la

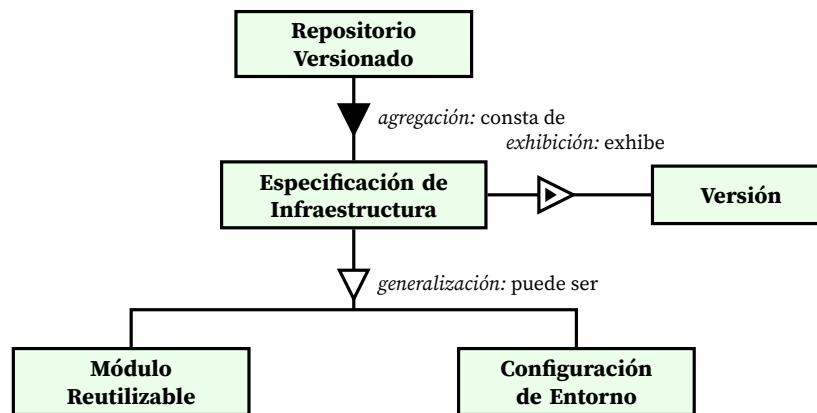


Figura 9: Enlaces estructurales del objeto *Especificación de Infraestructura*: agregación con el repositorio que la contiene, generalización en sus dos especializaciones y exhibición del atributo que la versiona. Elaboración propia.

diferencia entre desplegar y conciliar. Y tercero, que el equipo humano interviene sólo en dos de los siete subprocesos, lo que delimita el alcance real de la automatización.

10. Conclusiones

1. El funcionamiento de un sistema de infraestructura como código se reduce a un mecanismo único: la conciliación repetida entre un estado deseado, expresado en código versionado, y un estado real, observado a través de la interfaz de un proveedor. Todo lo demás, herramientas, formatos y flujos de trabajo, son variaciones sobre ese mecanismo.
2. La pieza que hace posible el mecanismo es el estado registrado. No es un detalle de implementación sino el elemento que permite reconocer la propiedad de los recursos, detectar destrucciones y encadenar dependencias, y por eso concentra los requisitos de cifrado, versionado y bloqueo.
3. Un sistema completo comprende siete capas, del sustrato del proveedor a la interfaz de autoservicio, y seis módulos transversales. Las capas intermedias compiten por el mismo territorio, de modo que la primera decisión de arquitectura consiste en asignar de forma explícita la propiedad de cada característica a una sola capa.
4. Los módulos transversales, y en particular la política como código y las pruebas, son los que distinguen un sistema que automatiza de uno que además garantiza. Su ausencia no impide desplegar: hace que los errores se desplieguen igual de rápido que los aciertos.
5. El tiempo de reconstrucción de un entorno completo desde el repositorio es la figura de mérito que mejor resume el estado de una implantación, porque verifica de una vez las cuatro propiedades que definen al sistema.
6. El modelado en OPM aporta algo que la descripción en prosa no aporta: obliga a declarar el tipo de cada relación. Al hacerlo aparecen con claridad la doble condición del registro de estado, la diferencia formal entre aplicar y conciliar, y el alcance exacto de la intervención humana, que en este modelo se limita a dos de los siete subprocesos.

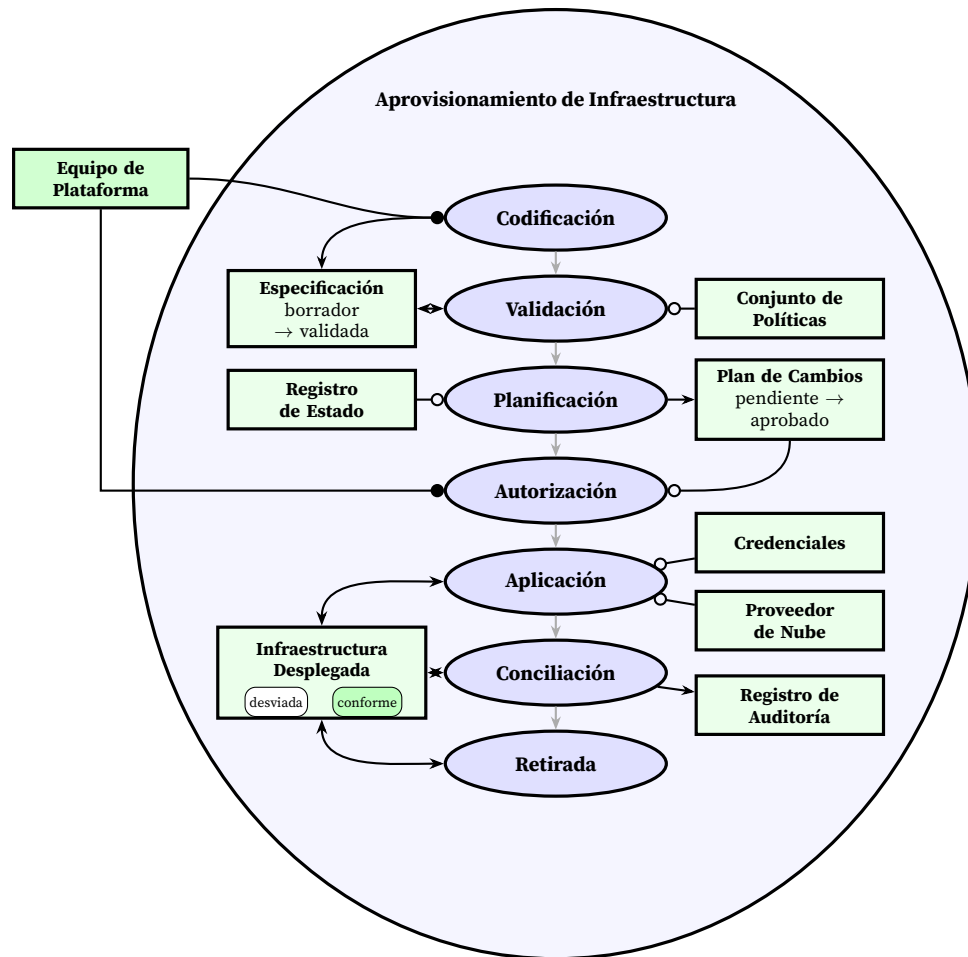


Figura 10: Diagrama SD1: ampliación en detalle del proceso de aprovisionamiento. La posición vertical de los subprocesos denota precedencia temporal. *Aplicación* y *Conciliación* actúan sobre el mismo objeto desde estados de partida distintos, que es la formulación precisa de la diferencia entre desplegar y corregir la deriva. El *Equipo de Plataforma* queda fuera del proceso ampliado por ser un objeto del entorno. Elaboración propia.

Referencias bibliográficas

- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50-57. <https://doi.org/10.1145/2890784>
- Dori, D. (2002). *Object-Process Methodology: A Holistic Systems Paradigm*. Springer. <https://doi.org/10.1007/978-3-642-56209-9>
- Dori, D. (2016). *Model-Based Systems Engineering with OPM and SysML*. Springer. <https://doi.org/10.1007/978-1-4939-3295-5>
- Enterprise Systems Modeling Laboratory, Technion. (s.f.). *OPCloud Sandbox*. Consultado el 27 de agosto de 2026, desde <https://opcloud-sandbox.web.app/>
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- HashiCorp. (s.f.). *Terraform State: Purpose*. Consultado el 27 de agosto de 2026, desde <https://developer.hashicorp.com/terraform/language/state/purpose>

- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- International Organization for Standardization. (2015, 15 de diciembre). *Automation Systems and Integration: Object-Process Methodology* (Especificación disponible al público ISO/PAS 19450:2015). ISO. Ginebra. Consultado el 27 de agosto de 2026, desde <https://www.iso.org/standard/62274.html>
- International Organization for Standardization. (2024, enero). *Automation Systems and Integration: Object-Process Methodology* (Norma internacional ISO 19450:2024). ISO. Ginebra. Consultado el 27 de agosto de 2026, desde <https://www.iso.org/standard/84612.html>
- Morris, K. (2021). *Infrastructure as Code: Dynamic Systems for the Cloud Age* (2.^a ed.). O'Reilly Media.
- Open Policy Agent. (s.f.). *Open Policy Agent Documentation*. Cloud Native Computing Foundation. Consultado el 27 de agosto de 2026, desde <https://www.openpolicyagent.org/docs/latest/>
- OpenGitOps. (s.f.). *GitOps Principles v1.0.0*. Cloud Native Computing Foundation. Consultado el 27 de agosto de 2026, desde <https://github.com/open-gitops/documents/blob/main/PRINCIPLES.md>
- Rahman, A., Mahdavi-Hezaveh, R., & Williams, L. (2019). A Systematic Mapping Study of Infrastructure as Code Research. *Information and Software Technology*, 108, 65-77. <https://doi.org/10.1016/j.infsof.2018.12.004>
- Rahman, A., Parnin, C., & Williams, L. (2019). The Seven Sins: Security Smells in Infrastructure as Code Scripts. *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 164-175. <https://doi.org/10.1109/ICSE.2019.00033>
- Red Hat. (s.f.). *Ansible Documentation: How Ansible Works*. Consultado el 27 de agosto de 2026, desde https://docs.ansible.com/ansible/latest/getting_started/index.html
- The Kubernetes Authors. (s.f.). *Kubernetes Documentation: Controllers*. Consultado el 27 de agosto de 2026, desde <https://kubernetes.io/docs/concepts/architecture/controller/>
- The Linux Foundation. (2023, 20 de septiembre). *Linux Foundation Launches OpenTofu: A New Open Source Alternative to Terraform*. Consultado el 27 de agosto de 2026, desde <https://www.linuxfoundation.org/press/announcing-opentofu>